

Decentralize Library dAPP

Subject: Blockchain

Members:

- **Mohd Sami - 61**
- **Ahad Shaikh - 53**
- **Mohd Sadik - 56**
- **Saurabh Sharma - 59**

DECENTRALIZED LIBRARY

1. PROJECT OVERVIEW & AIM

Project Title: Decentralized Library Using Blockchain

Aim: To design and implement a Decentralized Library System using Solidity, Remix IDE, MetaMask, Sepolia Test Network, and a web-based HTML frontend.

Key System Capabilities

- Add books to the blockchain.
- Borrow available books.
- Return borrowed books.
- View book details and current borrowing status.
- Deduct one library token from the user when a book is borrowed.

The project follows the assigned requirement that the Library contract depends on a Faucet smart contract for token deduction during borrowing.

2. SMART CONTRACT ARCHITECTURE

The project consists of two smart contracts :

1. **LibraryToken.sol** - Acts as the Faucet/Token contract to create tokens, store balances, allow the Library contract to spend tokens, and manage token transfers.
2. **DecentralizedLibrary.sol** - The main library contract handling book addition, storage, borrowing (with token deduction), returning, and details display.

2.1 LibraryToken.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract LibraryToken {

    string public name = "Library Token";

    string public symbol = "LIB";

    uint8 public decimals = 18;

    uint256 public totalSupply;

    address public owner;
```

```

mapping(address => uint256) public balanceOf;
mapping(address => mapping(address => uint256))
    public allowance;
event Transfer(
    address indexed from,
    address indexed to,
    uint256 value
);
event Approval(
    address indexed owner,
    address indexed spender,
    uint256 value
);
constructor() {
    owner = msg.sender;
}
modifier onlyOwner() {
    require(
        msg.sender == owner,
        "Only owner can perform this action"
    );
    _;
}
function mint(
    address to,
    uint256 amount

```

```

) public onlyOwner {
    require(to != address(0), "Invalid address");
    totalSupply += amount;
    balanceOf[to] += amount;
    emit Transfer(address(0), to, amount);
}

function transfer(
    address to,
    uint256 amount
) public returns (bool) {
    require(
        balanceOf[msg.sender] >= amount,
        "Insufficient balance"
    );
    balanceOf[msg.sender] -= amount;
    balanceOf[to] += amount;
    emit Transfer(msg.sender, to, amount);
    return true;
}

function approve(
    address spender,
    uint256 amount
) public returns (bool) {
    allowance[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}

```

```

}

function transferFrom(
    address from,
    address to,
    uint256 amount
) public returns (bool) {
    require(
        balanceOf[from] >= amount,
        "Insufficient balance"
    );
    require(
        allowance[from][msg.sender] >= amount,
        "Allowance exceeded"
    );
    allowance[from][msg.sender] -= amount;
    balanceOf[from] -= amount;
    balanceOf[to] += amount;
    emit Transfer(from, to, amount);
    return true;
}

function transferOwnership(
    address newOwner
) public onlyOwner {
    require(
        newOwner != address(0),
        "Invalid owner address"
    );
}

```

```

        );

        owner = newOwner;
    }
}

```

More Code on Github: <https://github.com/Sami21234/decentralized-library-blockchain-project>

3. DEPLOYMENT & SETUP IN REMIX IDE

1. **Compile LibraryToken.sol:** Select Solidity Compiler version 0.8.20 and compile.
2. **Connect MetaMask:** Set Environment to *Injected Provider - MetaMask* on the Sepolia Test Network.
3. **Deploy LibraryToken:** Enter initial supply as 1000000, deploy, confirm via MetaMask, and copy the contract address.
4. **Compile & Deploy DecentralizedLibrary:** Compile DecentralizedLibrary.sol (0.8.20), paste the LibraryToken address into the `_faucet` constructor parameter, and deploy.
5. **Link Contracts:** Call `setLibraryContract` on the deployed LibraryToken contract, passing the DecentralizedLibrary contract address.

4. FRONTEND IMPLEMENTATION

Project directory structure:
decentralized-library/

```

├── index.html
├── style.css
└── app.js

```

4.1 index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Decentralized Library</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
  <h1><img alt="Decentralized Library logo" data-bbox="208 885 228 898"/> Decentralized Library</h1>

```

```

    <p class="subtitle">Blockchain Based Library Management System</p>

    <button id="connectButton" onclick="connectWallet()">Connect
MetaMask</button>
    <p id="walletAddress">Wallet: Not Connected</p>
    <hr>

    <section>
        <h2>Add Book</h2>
        <input type="text" id="bookTitle" placeholder="Enter book
title">
        <button onclick="addBook()">Add Book</button>
    </section>
    <hr>

    <section>
        <h2>Borrow Book</h2>
        <input type="number" id="borrowBookId" placeholder="Enter Book
ID">
        <button onclick="borrowBook()">Borrow Book</button>
    </section>
    <hr>

    <section>
        <h2>Return Book</h2>
        <input type="number" id="returnBookId" placeholder="Enter Book
ID">
        <button onclick="returnBook()">Return Book</button>
    </section>
    <hr>

    <section>
        <h2>View Book</h2>
        <input type="number" id="getBookId" placeholder="Enter Book
ID">
        <button onclick="getBook()">View Book</button>
        <div id="bookDetails" class="book-card">No book
selected.</div>
    </section>

    <p id="message"></p>
</div>
<script
src="https://cdn.jsdelivr.net/npm/ethers@6.13.2/dist/ethers.umd.min.js
"></script>
<script src="app.js"></script>
</body>
</html>

```

4.2 style.css

```

body {
  font-family: Arial, sans-serif;
  background: #f4f6f8;
  margin: 0;
  padding: 30px;
}
.container {
  max-width: 650px;
  margin: auto;
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0,0,0,0.1);
}
h1 { text-align: center; margin-bottom: 5px; }
.subtitle { text-align: center; color: #666; }
section { margin: 25px 0; }
input {
  padding: 12px;
  width: 60%;
  border: 1px solid #ccc;
  border-radius: 6px;
  margin-right: 5px;
}
button {
  padding: 12px 18px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  background: #333;
  color: white;
  margin-top: 10px;
}
button:hover { opacity: 0.8; }
.book-card {
  margin-top: 20px;
  padding: 20px;
  background: #f5f5f5;
  border-radius: 8px;
  line-height: 1.7;
}
#message { text-align: center; font-weight: bold; margin-top: 20px; }

```

4.3 app.js

```

const LIBRARY_ADDRESS = "YOUR_LIBRARY_CONTRACT_ADDRESS";

const LIBRARY_ABI = [
  "function addBook(string title)",
  "function borrowBook(uint256 bookId)",
  "function returnBook(uint256 bookId)",

```

```

    "function getBook(uint256 bookId) view returns
(uint256,string,address,address,bool)",
    "function bookCount() view returns (uint256)"
];

let provider;
let signer;
let library;

async function connectWallet() {
  try {
    if (!window.ethereum) {
      alert("Please install MetaMask.");
      return;
    }
    provider = new ethers.BrowserProvider(window.ethereum);
    await provider.send("eth_requestAccounts", []);
    signer = await provider.getSigner();
    const address = await signer.getAddress();
    library = new ethers.Contract(LIBRARY_ADDRESS, LIBRARY_ABI,
signer);
    document.getElementById("walletAddress").innerText = "Wallet:
" + address;
    showMessage("MetaMask connected successfully.");
  } catch (error) {
    console.error(error);
    showMessage("Failed to connect MetaMask.");
  }
}

async function addBook() {
  if (!library) { alert("Connect MetaMask first."); return; }
  const title = document.getElementById("bookTitle").value.trim();
  if (!title) { alert("Please enter a book title."); return; }
  try {
    showMessage("Adding book...");
    const transaction = await library.addBook(title);
    await transaction.wait();
    showMessage("Book added successfully!");
    document.getElementById("bookTitle").value = "";
  } catch (error) {
    console.error(error);
    showMessage("Failed to add book.");
  }
}

async function borrowBook() {
  if (!library) { alert("Connect MetaMask first."); return; }
  const bookId = document.getElementById("borrowBookId").value;
  if (!bookId) { alert("Enter a Book ID."); return; }
  try {
    showMessage("Borrow transaction pending...");

```

```

        const transaction = await library.borrowBook(bookId);
        await transaction.wait(); async function returnBook() {
if (!library) { alert("Connect MetaMask first."); return; }
const bookId = document.getElementById("returnBookId").value;
if (!bookId) { alert("Enter a Book ID."); return; }
try {
    showMessage("Return transaction pending...");
    const transaction = await library.returnBook(bookId);
    await transaction.wait();
    showMessage("Book returned successfully!");
    document.getElementById("getBookId").value = bookId;
    await getBook();
} catch (error) {
    console.error(error);
    showMessage("Unable to return book.");
}
}

async function getBook() {
    if (!library) { alert("Connect MetaMask first."); return; }
    const bookId = document.getElementById("getBookId").value;
    if (!bookId) { alert("Enter a Book ID."); return; }
    try {
        const book = await library.getBook(bookId);
        document.getElementById("bookDetails").innerHTML = `
            <h3>Book Details</h3>
            <p><strong>Book ID:</strong> ${book[0]}</p>
            <p><strong>Title:</strong> ${book[1]}</p>
            <p><strong>Added By:</strong> ${book[2]}</p>
            <p><strong>Borrowed By:</strong> ${book[3]}</p>
            <p><strong>Status:</strong> ${book[4]} ? "  Borrowed" :
"  Available"</p>`;
        showMessage("Book details loaded.");
    } catch (error) {
        console.error(error);
        showMessage("Book not found or invalid Book ID.");
    }
}

function showMessage(message) {
    document.getElementById("message").innerText = message;
}

```

6. TEST CASE SUMMARY

Test Case	Objective	Description	Status
TC-01	Homepage / UI Load	Verify the decentralized library landing page and primary controls load correctly.	PASS
TC-02	Wallet Connection	Verify MetaMask connects and the account is visible.	PASS
TC-03	Sepolia Network	Verify the application and wallet are operating on Sepolia.	PASS
TC-04	Initial Balances	Verify LIB balance, faucet balance, resource count and network are displayed.	PASS
TC-05	Claim 5 LIB Tokens	Verify the faucet claim initiates a wallet transaction and completes successfully.	PASS
TC-06	Faucet Cooldown Protection	Verify a repeated claim is blocked during the cooldown period.	PASS
TC-07	Library Resource Loading	Verify decentralized resources are loaded with categories, costs and status.	PASS
TC-08	Resource Borrowing	Verify borrowed resources are represented in the library interface.	PASS
TC-09	Blockchain Transaction Verification	Verify the LIB transfer is confirmed on the Sepolia blockchain explorer.	PASS
TC-10	Borrowing History	Verify previously borrowed resources appear in the connected wallet's history.	PASS
TC-11	Borrowed Resource Reader	Verify a borrowed resource can be opened in the reading-room view.	PASS
TC-12	Wallet Disconnect / Reconnect	Verify the connected-account state can be cleared.	PASS

TC-01 - Homepage / UI Load

Test description: Verify that the decentralized library homepage loads correctly and presents the main navigation, wallet connection control, library exploration control, and dashboard cards.

Test procedure:

1. Open the decentralized library web application.
2. Observe the landing-page hero section.
3. Check the Connect Wallet and Explore Library controls.
4. Check the dashboard cards for LIB balance, faucet balance, available resources and network.

Expected result: The homepage loads without a blocking error and displays the main library interface and controls.

Observed result: The supplied screenshot shows the Decentralized Library landing page with the hero section, Connect Wallet button, Explore Library control and four dashboard cards.

Status: PASS

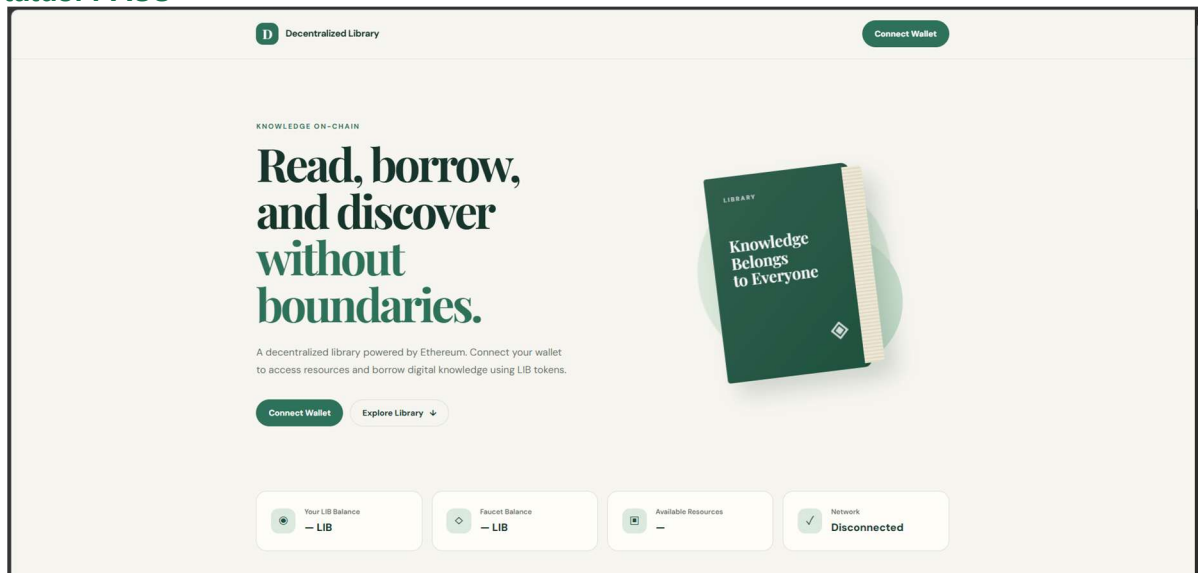


Figure TC-01 — Decentralized Library homepage.

TC-02 - MetaMask Wallet Connection

Test description: Verify that the web application can connect to the user's MetaMask wallet and that the wallet account is available for blockchain interaction.

Test procedure:

1. Open the application with MetaMask installed.
2. Select Connect Wallet.
3. Approve the connection request in MetaMask.
4. Check the connected account shown by MetaMask.

Expected result: The wallet connection is established and the selected account is available for application transactions.

Observed result: MetaMask is open for Account 1 and shows a wallet address ending in EA336. The activity view also identifies the Sepolia network.

Status: PASS

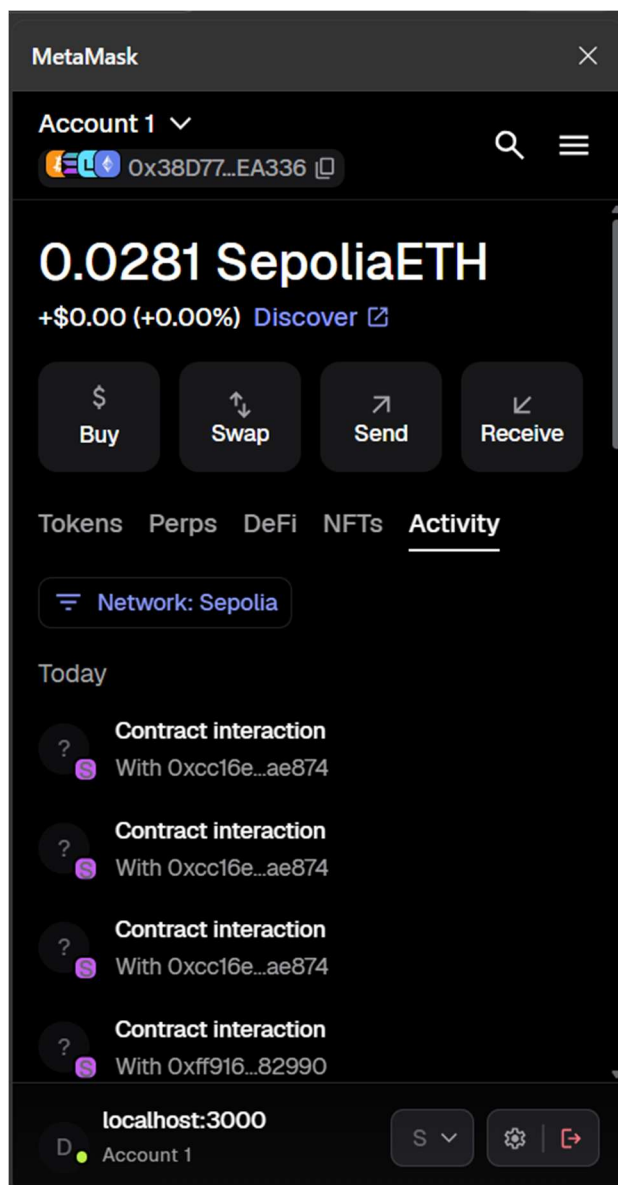


Figure TC-02 — MetaMask account connected to the application.

TC-03 - Sepolia Network Verification

Test description: Verify that the application and connected wallet are operating on the Sepolia test network used by the project.

Test procedure:

1. Connect the MetaMask wallet.
2. Observe the network shown in the application dashboard.
3. Observe the network shown in MetaMask.

Expected result: The application and wallet both show Sepolia, allowing the project contracts and test tokens to be used on the intended test network.

Observed result: The screenshot shows 'Sepolia' in the application network card and 'Network: Sepolia' inside MetaMask.

Status: **PASS**

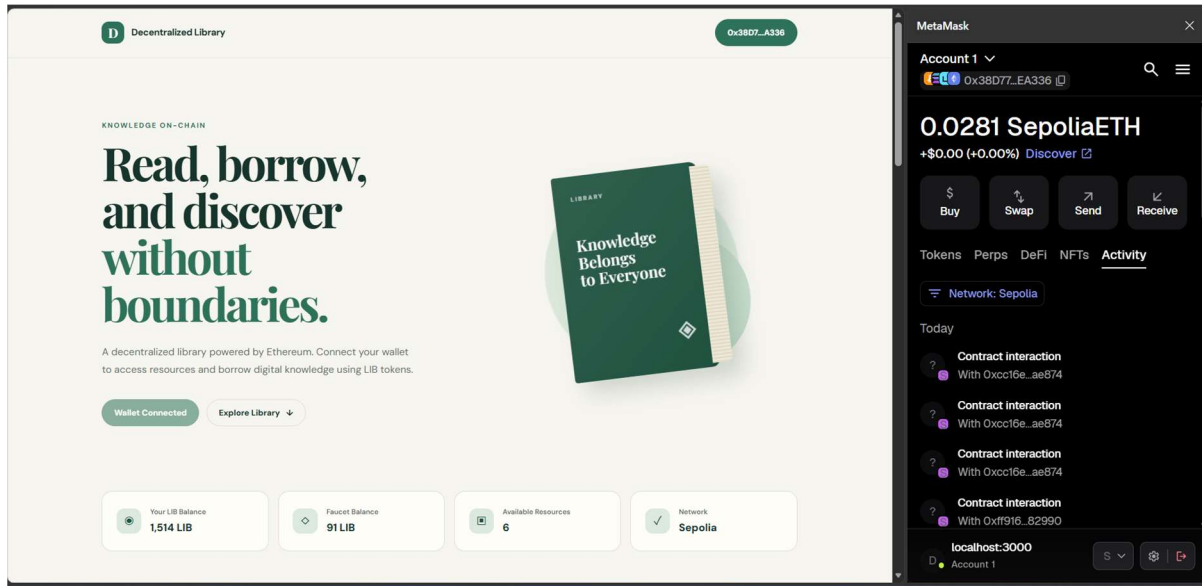


Figure TC-03 — Sepolia network shown in the application and MetaMask.

TC-04 - Initial Balances and Resource Count

Test description: Verify that the dashboard displays the connected wallet's LIB balance, faucet balance, available resource count and current network.

Test procedure:

1. Connect the wallet.
2. Observe the dashboard cards.
3. Record the displayed LIB balance, faucet balance, available resources and network.

Expected result: The dashboard displays non-empty blockchain/application state values for the connected account.

Observed result: The screenshot shows 1,514 LIB as the user's LIB balance, 91 LIB as the faucet balance, 6 available resources and Sepolia as the network.

Status: PASS

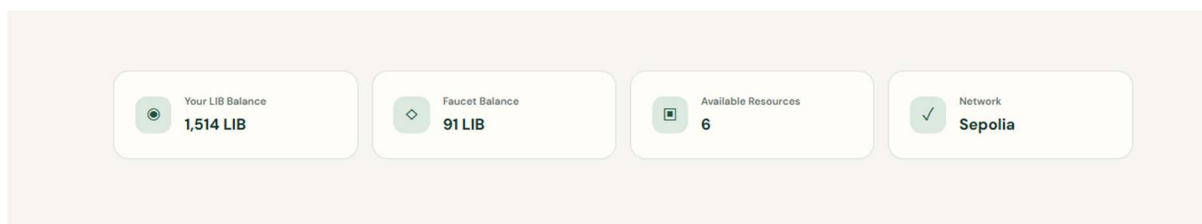


Figure TC-04 — Initial dashboard balances and resource count.

TC-05 - Claim 5 LIB Tokens

Test description: Verify that the faucet allows the connected wallet to request 5 LIB tokens and that the transaction is confirmed by MetaMask.

Test procedure:

1. Open the Need LIB tokens section.

2. Select Claim 5 LIB.
3. Review the MetaMask transaction request.
4. Confirm the transaction in MetaMask.
5. Return to the application and observe the claim result.

Expected result: A wallet transaction is requested and, after confirmation, the application reports that 5 LIB tokens were sent to the wallet.

Observed result: The first evidence image shows the MetaMask transaction request for receiving +5 LIB on Sepolia. The second image shows 'Claimed 5 LIB' and the success message stating that 5 LIB tokens were sent to the wallet.

Status: PASS

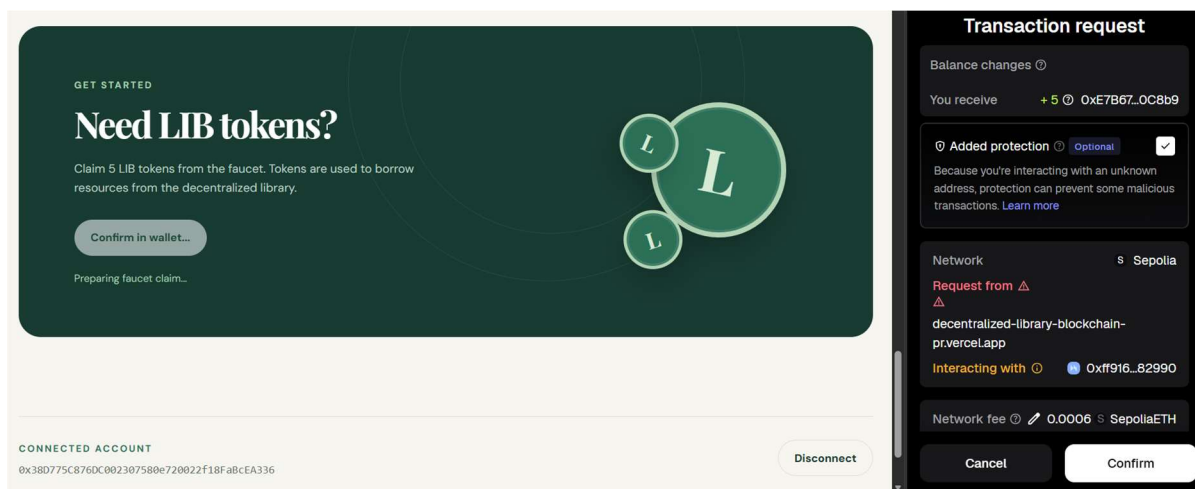


Figure TC-05A — LIB faucet claim transaction request.

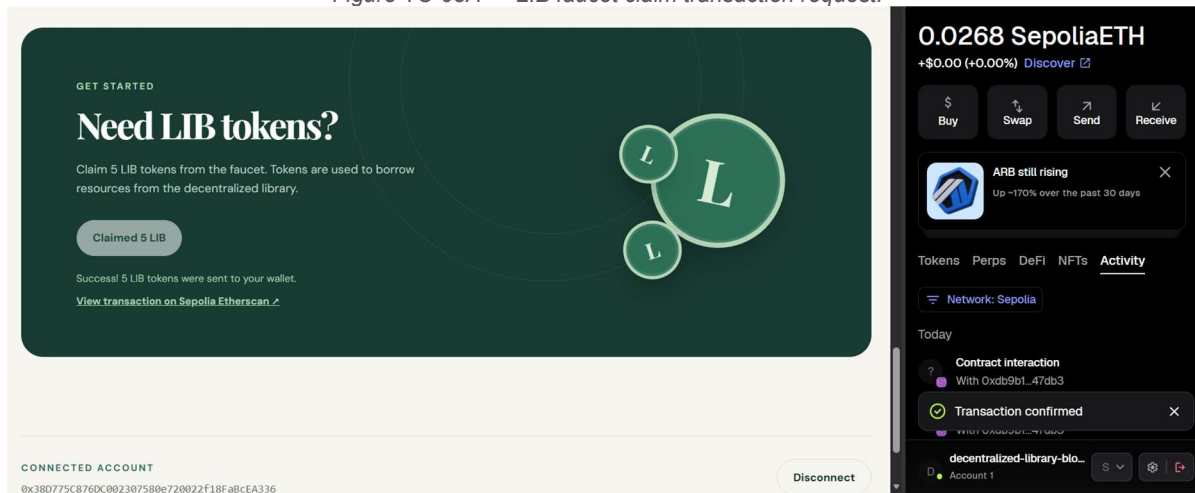


Figure TC-05B — Successful 5 LIB claim confirmation.

TC-06 - Faucet Cooldown Protection

Test description: Verify that the faucet prevents an immediate repeated claim after a successful claim.

Test procedure:

1. Claim 5 LIB successfully.

2. Immediately attempt to claim 5 LIB again.

3. Observe the message displayed by the faucet section.

Expected result: The second claim is blocked and a cooldown message is displayed instead of issuing another immediate claim.

Observed result: The screenshot shows the Claim 5 LIB button and the message 'Please wait before claiming again'.

Status: PASS

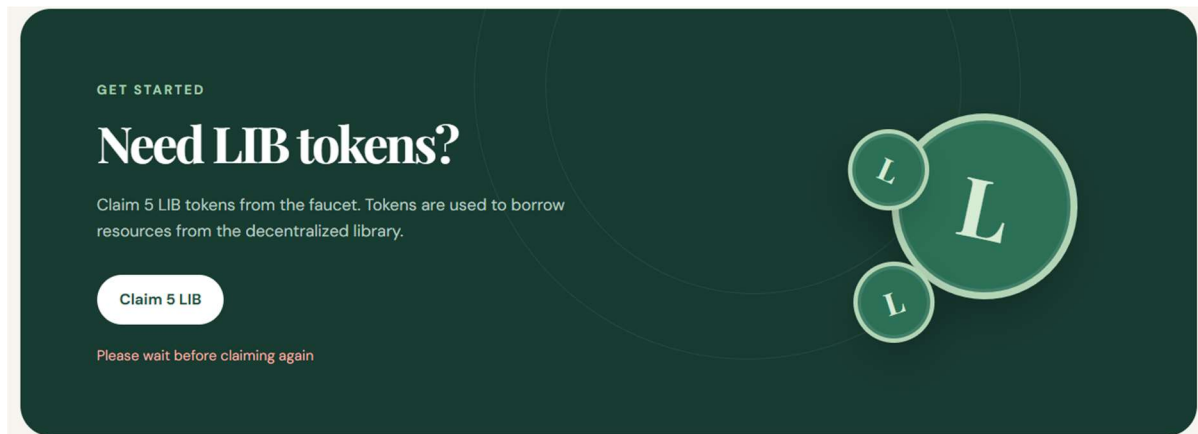


Figure TC-06 — Faucet cooldown protection after a claim.

TC-07 - Library Resource Loading

Test description: Verify that the Explore the Library section loads the available decentralized resources with their categories, titles, descriptions, costs and borrowing state.

Test procedure:

1. Scroll to the Explore the Library section.

2. Observe the resource cards.

3. Check the category, title, description, cost and current borrowing message.

Expected result: Library resources are rendered correctly and the interface provides a way to open/read each resource.

Observed result: The screenshot displays Blockchain Fundamentals, Smart Contract Design and The Future of Libraries with categories, descriptions, a cost of 1 LIB and Read controls.

Status: PASS

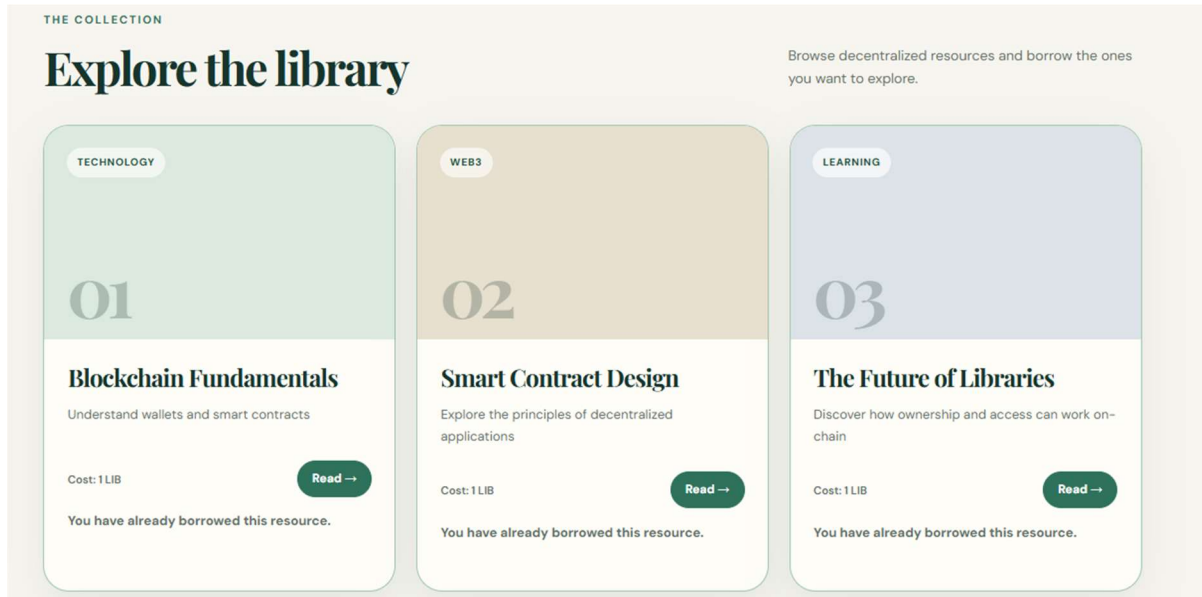


Figure TC-07 — Library resource cards loaded successfully.

TC-08 - Resource Borrowing

Test description: Verify that resources are represented as borrowable library items and that the interface reflects the borrowed state for resources associated with the connected wallet.

Test procedure:

1. Open the library resource collection.
2. Select a resource through its Read/borrow control.
3. Observe the resource card after borrowing.
4. Verify the card reflects the borrowed state.

Expected result: A successfully borrowed resource is shown as already borrowed by the connected wallet and can be opened through the available Read control.

Observed result: The supplied screenshot shows resources 05 and 06 with 'You have already borrowed this resource.' and active Read controls.

Status: PASS

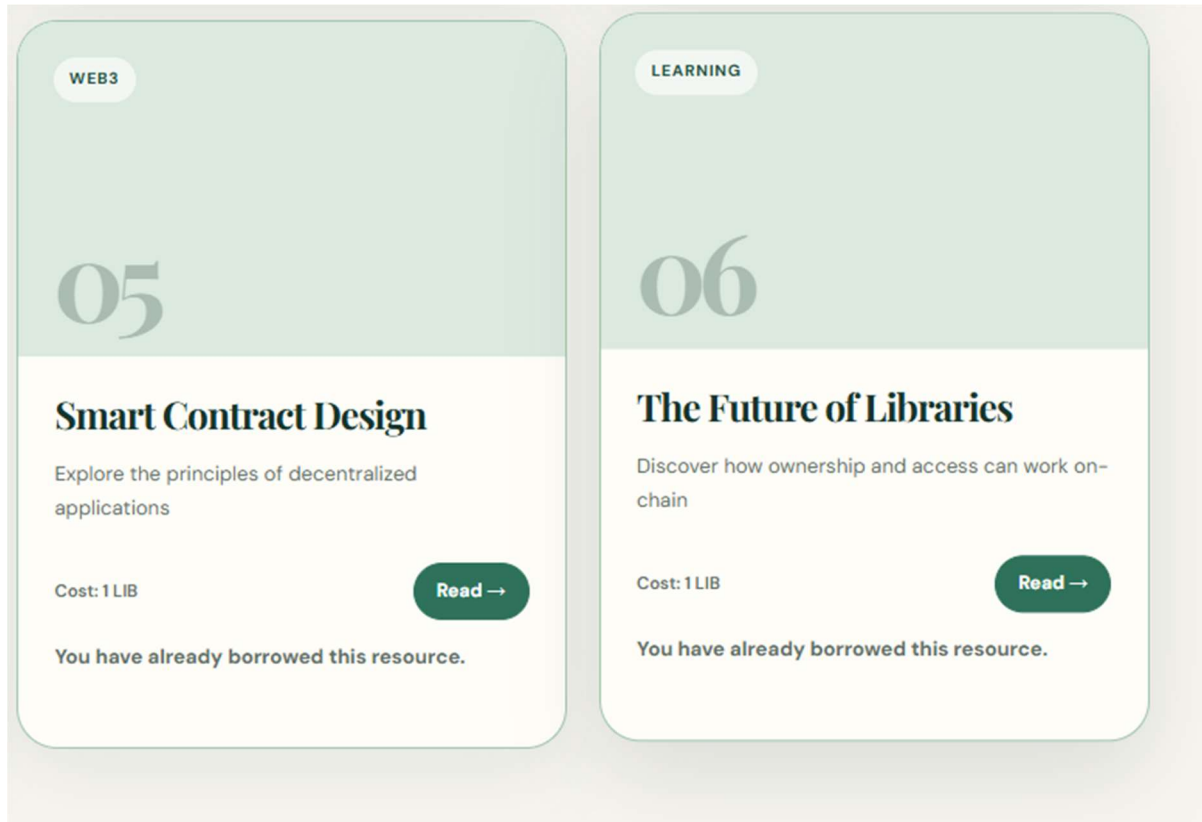


Figure TC-08 — Resource cards showing the borrowed state.

TC-09 - Blockchain Transaction Verification

Test description: Verify that the token transfer generated by the faucet claim is recorded and confirmed on the Sepolia blockchain explorer.

Test procedure:

1. Complete the 5 LIB faucet claim.
2. Open the transaction link in the Sepolia blockchain explorer.
3. Check the transaction status and token transfer details.

Expected result: The explorer should show a successful Sepolia transaction with the expected LIB token transfer.

Observed result: The supplied explorer screenshot shows a Sepolia test transaction with Status: Success and an ERC-20 transfer of 5 LIB to the connected wallet.

Status: **PASS**

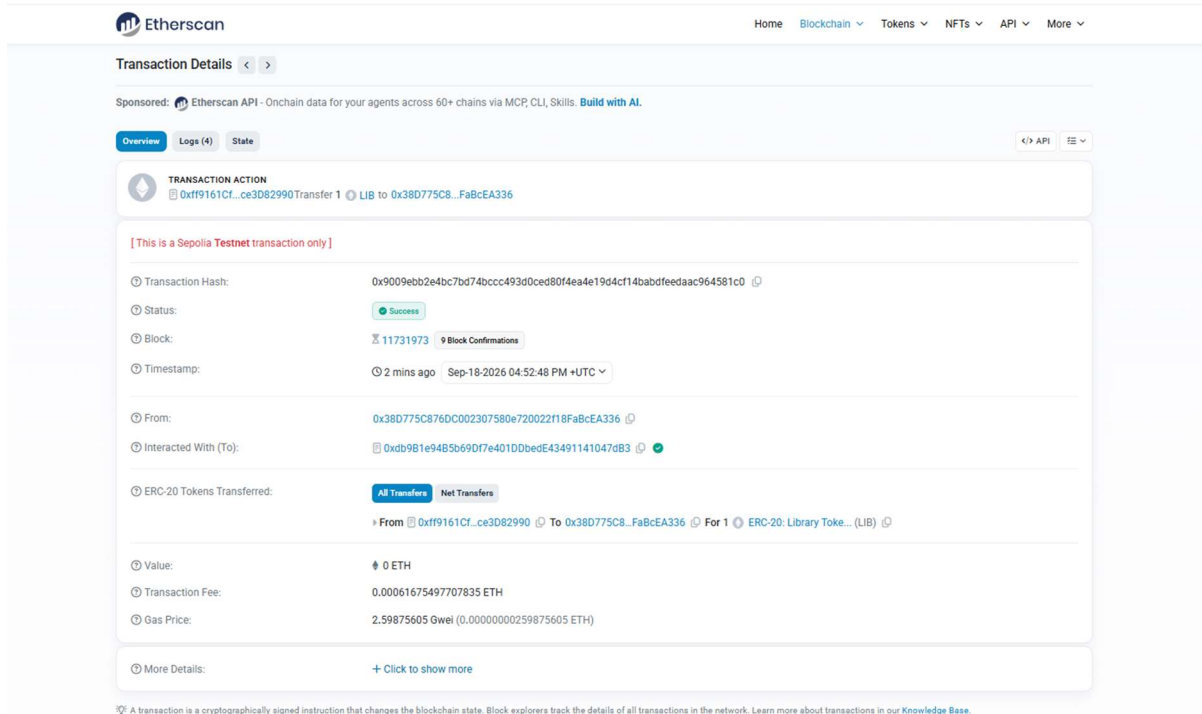


Figure TC-09 — Successful LIB token transfer verified on the Sepolia explorer.

TC-10 - Borrowing History

Test description: Verify that the connected wallet's previously borrowed resources are displayed in the Borrowing history section.

Test procedure:

1. Connect the wallet.
2. Navigate to the Borrowing history section.
3. Review the listed resources and their Read controls.

Expected result: Previously borrowed resources are listed with their titles/categories and can be opened from the history.

Observed result: The screenshot shows a Borrowing history list containing multiple resources, including Blockchain Fundamentals, Smart Contract Design and The Future of Libraries, each with a Read control.

Status: PASS

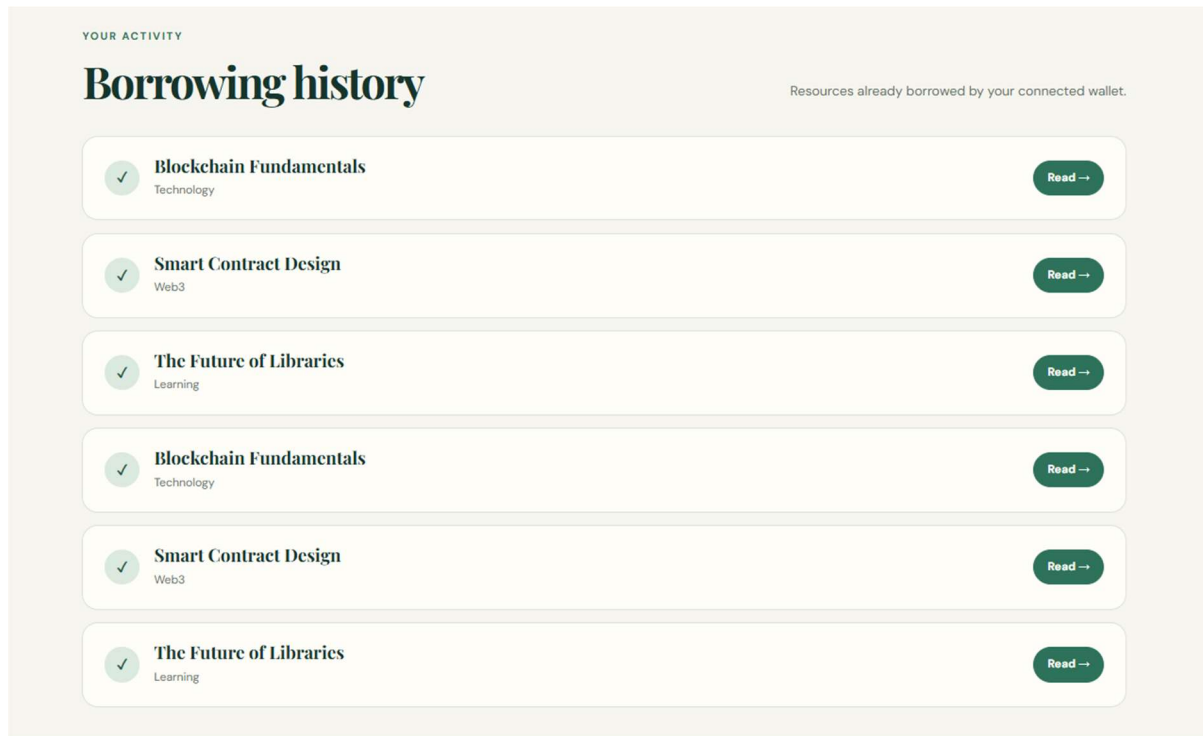


Figure TC-10 — Borrowing history for the connected wallet.

TC-11 - Borrowed Resource Reader

Test description: Verify that a borrowed library resource opens in the decentralized reading-room interface and displays its content.

Test procedure:

1. Open a borrowed resource using Read.
2. Observe the reading-room modal.
3. Check the resource title and body content.

Expected result: The selected resource opens in a readable modal/view with its title and content.

Observed result: The screenshot shows the 'DECENTRALIZED READING ROOM' with the title 'Smart Contract Design' and explanatory content about smart contracts and the library/faucet design.

Status: **PASS**

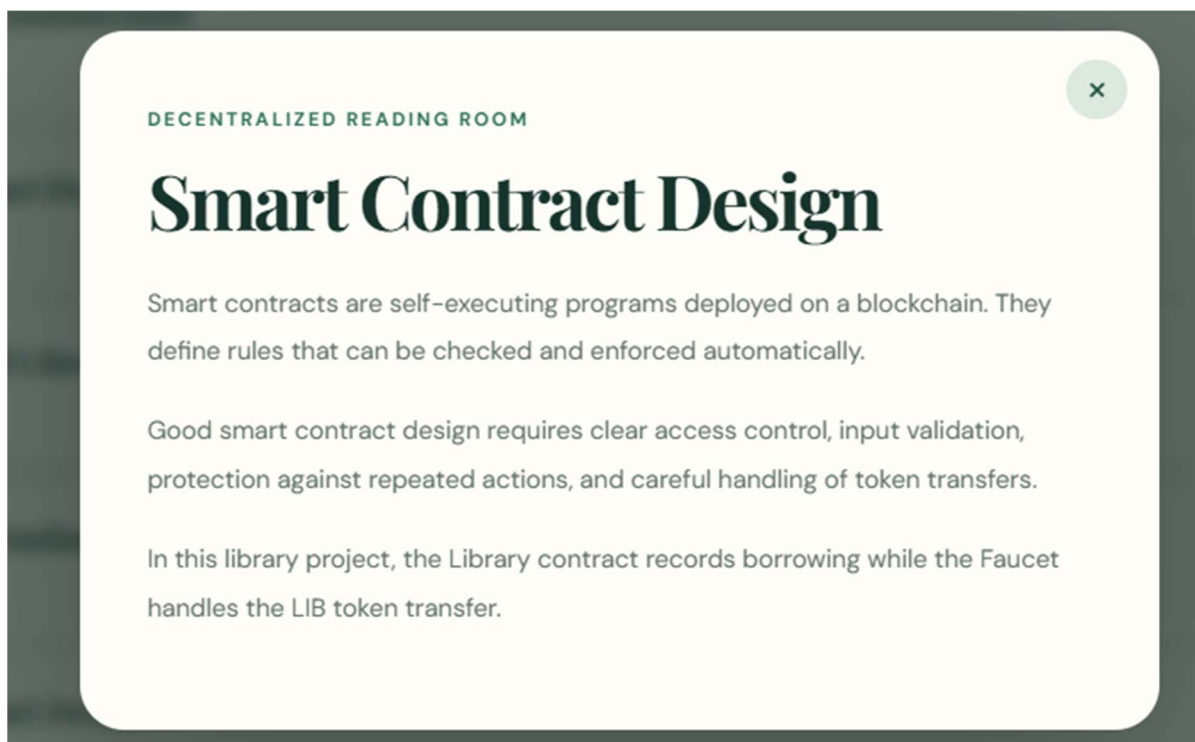


Figure TC-11 — Borrowed resource opened in the decentralized reading room.

TC-12 - Wallet Disconnect / Reconnect

Test description: Verify that the connected-account state can be cleared from the application after wallet interaction.

Test procedure:

1. Connect the MetaMask wallet.
2. Use the Disconnect control in the connected-account section.
3. Observe the account state after disconnection.

Expected result: The application should no longer display an active connected wallet address after disconnecting.

Observed result: The supplied screenshot shows the Connected Account section with 'No wallet connected' and the Disconnect control, confirming the disconnected state is represented in the interface.

Status: PASS

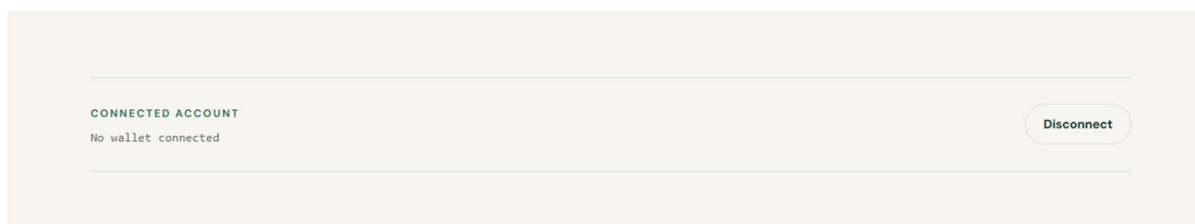


Figure TC-12 — Wallet disconnected state.

7. OVERALL VALIDATION RESULT

Result: PASS - all supplied screenshot-based test cases are documented with evidence.

The evidence demonstrates the main end-to-end flow of the decentralized library application: opening the interface, connecting MetaMask, using the Sepolia test network, viewing balances, claiming LIB tokens through the faucet, enforcing the faucet cooldown, loading library resources, representing borrowed resources, verifying an on-chain LIB transfer, viewing borrowing history, opening borrowed content, and displaying the disconnected-wallet state.

8. TEST EVIDENCE INDEX

Test Case	Evidence File	Validation Area
TC-01	TC-01-homepage.png	Homepage and dashboard
TC-02	TC-02-wallet-connected.png	MetaMask wallet connection
TC-03	TC-03-sepolia-network.png	Sepolia network
TC-04	TC-04-initial-balances.png	Balances and resource count
TC-05	TC-05-get_token.png + TC-05-get_token_confirmation.png	Faucet claim and confirmation
TC-06	TC-06 Faucet Cooldown Protection.png	Cooldown protection
TC-07	TC-07 Library Resource Loading.png	Library resource loading
TC-08	TC-08 Resource Borrowing.png	Borrowed resource state
TC-09	TC-09 Blockchain Transaction Verification.png	On-chain transaction verification
TC-10	TC-10 Borrowing History.png	Borrowing history
TC-11	TC-11 Borrowed Resource Reader.png	Resource reading interface
TC-12	TC-12 Wallet Disconnect and Reconnect.png	Disconnected wallet state